

Miniproject viva

Symmetric Key Encryption

- * Uses one single key
- * Same key is used to lock (encrypt) and unlock (decrypt) data

👉 Example:

You and your friend both have the same password to open a box.

Pros:

- * Faster ⚡
- * Easy to use

Cons:

- * You must share the key safely (risk if someone steals it)

🔑 Asymmetric Key Encryption

- * Uses two keys:
 - * Public key (can share with everyone)
 - * Private key (keep secret)

👉 Example:

Anyone can lock a box using your public key, but only you can open it with your private key.

Pros:

- * More secure 
- * No need to share private key

Cons:

- * Slower than symmetric

Feature	Symmetric Key.	Asymmetric Key
Keys used.	1 key	2 keys
Speed	Fast 	Slower 
Security	Less secure	More secure
Key sharing	Required	Not required

Easy Trick to Remember

- * Symmetric = Same key
- * Asymmetric = Different keys

DES

it is a type of symmetric key encryption.

👉 That means:

- * Same key is used to lock and unlock data

💡 Easy Example

Think of DES like a lock and key system:

- * You lock your message using a secret key 🔑
- * The receiver uses the same key to open it

⚙️ How DES works (very simple)

- * Takes data in blocks (64 bits)
- * Uses a secret key (56 bits)

DES works like this:

1 Take input

- * Take 64-bit data (message)
- * Take 56-bit secret key

2 Initial Permutation (IP)

- * Rearrange (shuffle) the bits of data

👉 Just changing positions, not values

3 Split data

* Divide into 2 parts:

* Left (L)

* Right (R)

4 Do 16 rounds (main part)

For each round:

* Use a round key (generated from main key)

* Apply function on Right part:

* Expansion

* XOR with key

* Substitution (S-box)

* Permutation

* Swap Left and Right

 This repeats 16 times

5 Final step

* Combine Left and Right

* Apply Final Permutation

Output

* You get encrypted data (cipher text)

 Super short memory trick:

IP → Split → 16 Rounds → Combine → FP

Advantages

- * Fast 
- * Easy to implement

Disadvantages

- * Not very secure today 
- * Key is too small → easy to crack with modern computers

One-line definition

DES encrypts data by splitting it into two parts and processing it through 16 rounds using a secret key.

What is AES?

Advanced Encryption Standard (AES) is a secure encryption method used to protect data.

 It converts plain text (normal data) into cipher text (secret data) so no one can read it without the key.

Where is AES used?

- * WhatsApp messages 
- * Online banking 
- * Wi-Fi passwords 
- * File encryption 

 Basically everywhere security is needed

How AES works (easy steps)

Input

- * Data block = 128 bits
- * Key = 128 / 192 / 256 bits

Rounds (main process)

AES works in multiple rounds:

- * 10 rounds → 128-bit key
- * 12 rounds → 192-bit key
- * 14 rounds → 256-bit key

Each round has these steps 

Steps inside each round

1. SubBytes

- * Replace bytes using a table (S-box)

2. ShiftRows

- * Shift rows left (rearrange data)

3. MixColumns

- * Mix data mathematically

4. AddRoundKey

- * Combine data with key using XOR

Final Round

- * Same steps but no MixColumns

Output

- * You get encrypted data (cipher text)

To decrypt → reverse all steps using the key

 AES vs S-AES (quick idea)

 AES vs S-AES (quick idea)		
Feature	AES	S-AES
Data size	128 bits	16 bits
Key size	128/192/256	16 bits
Rounds	10–14	2
Usage	Real world	Learning

 Important points

- * AES is very secure and fast
- * It is a symmetric key algorithm (same key for encrypt & decrypt)
- * Adopted as a standard by National Institute of Standards and Technology

 Easy memory trick

 “SSMA”

- * S → SubBytes
- * S → ShiftRows

* M $\rightarrow$ MixColumns

* A $\rightarrow$ AddRoundKey

Diffie–Hellman Key Exchange (Explanation)

Diffie–Hellman key exchange is a method used to **securely share a secret key between two parties over an insecure channel**.

It was proposed by Whitfield Diffie and Martin Hellman in 1976.

How it works:

First, both users agree on two public values:

- A large prime number **p**
- A base (generator) **g**

Then,

- User A selects a private key **a**
- User B selects a private key **b**

They compute public values:

- A computes **$A = g^a \bmod p$**
- B computes **$B = g^b \bmod p$**

These values are exchanged openly.

Secret Key Generation:

- A computes **$K = B^a \bmod p$**
- B computes **$K = A^b \bmod p$**

Both get the **same shared secret key**, which is used for encryption.

Security:

The security is based on the **Discrete Logarithm Problem**, which makes it very difficult to find the private key from the public values.

Limitation:

It is vulnerable to a **man-in-the-middle attack** if authentication is not used.

Conclusion:

👉 Diffie–Hellman allows two users to **securely generate a common key without directly transmitting it**, making it very useful in secure communication systems.

RSA (in simple words) Rivest–Shamir–Adleman

RSA is a method used to **secure messages** using **two keys**:

-  **Public key** → used to **lock (encrypt)**
-  **Private key** → used to **unlock (decrypt)**

It was created by:

- Ron Rivest
 - Adi Shamir
 - Leonard Adleman
-

Easy Example

Think of a **lock and key system**:

1. You give everyone an **open lock (public key)** 
2. Anyone can put a message in a box and **lock it**
3. Only you have the **key (private key)**  to open it

👉 So only **you can read the message**

How it works (simple steps)

1. Choose two big prime numbers
 2. Do some math to create:
 - Public key
 - Private key
 3. Sender uses **public key** to encrypt
 4. Receiver uses **private key** to decrypt
-

Why it's secure

Because it's very hard to break large numbers into prime factors

👉 Based on Integer Factorization Problem

2. Key Generation

1. Choose two large prime numbers **p** and **q**
2. Compute **n = p × q**
3. Compute **φ(n) = (p - 1)(q - 1)** ... Euler's Totient Function
4. Choose public key **e** such that:
 - $1 < e < \phi(n)$
 - $\gcd(e, \phi(n)) = 1$
5. Compute private key **d** such that:
 - $d \times e \equiv 1 \pmod{\phi(n)}$

👉 Public key = (e, n)

👉 Private key = (d, n)

3. Encryption

- Sender converts message into number **M**
- Ciphertext is calculated as:

$$C = M^e \pmod{n}$$

4. Decryption

- Receiver uses private key:

$$M = C^d \pmod{n}$$

5. Security

- Based on difficulty of ****Integer Factorization Problem**
- Hard to find p and q from n

6. Important Points

- Uses **two keys**:
 - Public key $\rightarrow$ encryption
 - Private key $\rightarrow$ decryption
- Keys are mathematically related
- Widely used in secure systems

7. Applications

- Secure data transmission
- Digital signatures
- SSL/TLS protocols

Short Definition (Exam Ready)

RSA is an asymmetric encryption algorithm that uses a public key for encryption and a private key for decryption.

Elliptic Curve Cryptography (ECC) — in simple language

ECC is a modern encryption method used to **secure data and create keys**, just like RSA—but it uses **different math (elliptic curves)** and achieves the **same security with much smaller keys**.

💡 Simple idea

Instead of using big-number multiplication (like RSA), ECC uses **points on a special curve**.

👉 Think of it like:

- You pick a **secret number**
 - You move along a curve using that number
 - You share the **resulting point**, not your secret
 - ➡ Others see the point, but **can't find your secret number**
-

📐 What is an Elliptic Curve?

ECC uses equations like:

$$y^2 = x^3 + ax + b$$

This curve has **many points**, and we do math with those points.

🧠 Key Concepts (very easy)

- **Curve** → the equation above
- **Point (G)** → a fixed starting point
- **Private key** → a secret number (like 5, 10, etc.)
- **Public key** → result of multiplying G by private key

👉 Public key = private key × G

⚙️ Working (step-by-step)

1 Agree on public values

- Curve equation
 - Base point G
-

2 Choose secrets

- Alice chooses **a (private key)**
- Bob chooses **b (private key)**

Create public keys

- Alice $\rightarrow A = a \times G$
- Bob $\rightarrow B = b \times G$

They exchange A and B

Create shared secret

- Alice $\rightarrow S = a \times B$
- Bob $\rightarrow S = b \times A$

 Both get **same secret key** 

Why ECC is secure

Because of a hard problem called:

 Elliptic Curve Discrete Logarithm Problem

It's very hard to find the private key from the public key.

Why ECC is better than RSA

Feature	ECC	RSA
Key size	Small 	Large 
Speed	Faster 	Slower
Security	High	High

 Example:

ECC 256-bit $\approx$ RSA 3072-bit security

Where ECC is used

- HTTPS (secure websites)
- Mobile apps
- Cryptocurrencies like Bitcoin
- Messaging apps

Short Definition (Exam Ready)

ECC is an asymmetric encryption technique that uses elliptic curves to generate public and private keys, providing high security with smaller key sizes.